사내 AI 해커톤 · 서류 심사 제출서

contune
배치 실행부터 원인 함수까지, 번호 하나로 잇는 운영 도구

지금은 **배치 실행**, **서버 자원**, **업무 처리 현황**을 서로 다른 도구 3개가 따로 봅니다. 장애가 나면 담당자가 세 화면의 시각을 맞춰 가며 원인을 찾고, 원인 코드는 같은 부하를 다시 만들어야 겨우 찾습니다.

contune은 세 도구가 하던 일을 실행 파일 하나로 새로 만들고, 실행마다 **실행 번호**를 붙여 자원·업무 지표·함수 기록까지 같은 번호로 우리 DB에 쌓습니다.

3 → 1
운영 도구 수
스케줄러·서버 관제·업무 관제를 하나로

5 → 2
장애 원인 추적 단계
지나간 장애도 재현 없이 함수까지
시연 시나리오 기준

0.6%
함수 추적 부하
코어 1개 기준 프로파일러 CPU
사전 실측(C 시험 프로그램)

심사 기준	이 제안의 답	근거
창의성 30	스케줄러가 발급한 실행 번호를 프로세스에 심어, 실행·업무 지표·자원·**CPU를 쓴 함수**를 한 키로 잇습니다. 모든 데이터는 공개 스키마로 우리가 소유합니다.	3장, 6장
업무 도구 실용성 30	운영에서 매일 쓰는 기능(정해진 시각 실행, 선후행, 재시도, 원격 실행, 서버·업무 관제)만 실행 파일 하나에 담았습니다. 방화벽은 매니저 포트 1개입니다.	5장, 6장
구현 가능성 20	범위를 개념 7개로 못 박고, 2명이 AI 세션 4개로 하루에 만듭니다. 함수 추적의 핵심 경로와 부하는 미리 실측했습니다.	7장
기대 효과 20	원인 추적 시간 단축, 운영 데이터 5종의 분석 활용, 상용 라이선스 비용 절감.	8장

01 우리 운영 환경 한눈에 보기

외부 위원께서 이후 내용을 읽으실 수 있도록 운영 환경부터 짧게 설명합니다. 우리 업무의 중심은 **주식·파생상품 주문을 받아 체결하는 온라인 업무**입니다. 배치는 장 시작 전 준비와 장 마감 후 정산에 씁니다.

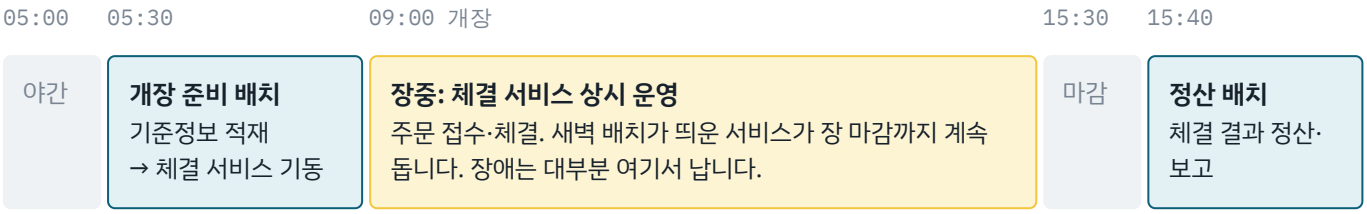


그림 1 하루 운영 흐름. 배치와 온라인은 따로 떨어져 있지 않습니다. 장중 서비스를 띄우는 것도 새벽 배치입니다.

같은 사건을 도구 3개가 나눠 봅니다

배치 실행, 서버 자원, 업무 처리 현황을 각각 다른 도구가 맡고 있습니다. 도구마다 화면과 저장소가 따로 있고, **세 도구가 함께 쓰는 공통 키가 없습니다.** 어느 코드가 CPU를 썼는지(함수 단위)는 맡는 도구가 아예 없습니다.

배치 스케줄러 상용 제품 (Control-M)	서버 관제 상용 제품 (onTune)	업무 관제 사내 개발 APM (CS 프로그램)
무엇을, 언제 실행했고 성공했는가 `단위: 잡`	어느 서버, 어느 프로세스가 CPU·메모리를 썼는가 `단위: 서버 · 프로세스 번호`	주문이 초당 몇 건 들어오고 처리됐으며, 응답이 얼마나 걸렸는가 `단위: 업무 프로세스`
자체 화면 · 자체 저장소	자체 화면 · 자체 저장소	자체 화면 · 자체 저장소

공통 키 없음 → 사람이 시각과 서버 이름으로 맞춤

어느 **함수(코드)**가 CPU를 썼는가 → 맡는 도구 없음. 필요할 때 개발자가 서버에 접속해 직접 측정

그림 2 지금의 운영 도구 구성

이 문서에 나오는 용어

배치 · 잡	정해진 시각에 자동으로 실행하는 작업. 운영에서는 셸 스크립트이고, 스크립트가 C로 만든 서비스 프로그램을 띄웁니다.
플로우(DAG)	순서가 있는 잡 묶음. "A가 끝나면 B와 C를 동시에, 둘 다 끝나면 D" 같은 선후행을 화살표 그래프로 나타냅니다.
실행 번호	스케줄러가 실행 1회마다 발급하는 고유 번호(`exec_id`). 택배 송장번호처럼 그 실행에서 나온 모든 기록에 붙습니다.
p99 응답시간	주문 100건 중 가장 느린 1건 수준의 처리 시간. 체결이 밀리는지 보는 대표 지표입니다.
플레임 그래프	프로그램이 CPU를 어느 함수에 썼는지를 막대 폭으로 보여 주는 그림. 폭이 넓을수록 CPU를 많이 쓴 함수입니다.

02 무엇이 문제인가

문제 1

특이상황을 확인하기 번거롭다

도구 3개를 오가며 시각을 맞춥니다. 몇 분 만에 지나간 장애는 원인 코드를 다시 볼 수 없습니다.

문제 2

쌓인 데이터를 활용하기 어렵다

데이터가 제품마다 따로 쌓여 서로 엮을 수 없습니다. 통계나 머신러닝에 쓸 원자료를 우리가 쥐고 있지 않습니다.

문제 3

비용이 많이 든다

상용 제품 2개의 라이선스·유지보수와 사내 APM 유지보수를 따로 냅니다. 쓰지 않는 기능 값도 함께 냅니다.

문제 1. 장중 주문 폭주 한 건을 추적하는 지금의 순서

예를 들어 장중에 주문이 몰려 체결 처리가 밀렸다고 하겠습니다. 이런 폭주는 보통 몇 분 만에 지나갑니다.

지금 — 도구 3개 + 사람 · 5단계

1	업무 관제	응답시간이 오르고 처리 대기 건수가 쌓인 것을 확인
2	운영 담당자	그 체결 서비스가 어느 서버의 어느 프로세스인지 운영 문서와 담당자에게 확인
3	서버 관제	그 서버, 그 시각을 찾아 CPU 확인. **"체결 프로세스가 CPU를 다 썼다"에서 멈춤**
4	배치 스케줄러	그 서비스를 띄운 실행과 같은 시각에 돈 다른 작업을 확인
5	개발자	원인 코드를 찾으려 하지만 폭주는 이미 끝남. **같은 부하를 다시 만들어 재현**한 뒤 측정

contune — 화면 1개 · 2단계

1. 대시보드의 응답시간 타일을 누르면, 그 서비스를 띄운 실행의 **타임머신**이 열립니다. 업무 지표·서비스 자원·서버 자원이 한 시간축에 겹쳐 있습니다.
2. 차트에서 밀린 구간을 끌어 고르면 그 구간에 **CPU를 쓴 함수**가 플레임 그래프로 나옵니다. 지나간 구간이어도 됩니다.

찾을 필요가 없는 이유: 업무 지표에 이미 그 서비스를 띄운 `실행 번호`가 붙어 있습니다.

그림 3 같은 장애의 원인을 찾는 순서 비교. 가장 큰 손실은 5단계입니다. 지나간 장애는 다시 볼 수 없어 재현부터 해야 합니다.

문제 2. 데이터는 쌓이는데 질문에 답하지 못합니다

운영에서 정말 궁금한 질문은 거의 다 **두 도구 이상의 데이터를 엮어야** 답할 수 있습니다. 지금은 데이터가 제품마다 따로 쌓이고, 저장 형식과 보존 기간, 꺼낼 수 있는 범위를 제품이 정합니다. 결국 제품이 보여 주는 화면과 보고서 안에서만 보게 됩니다.



그림 4 따로 쌓인 데이터로는 답할 수 없는 질문들. 통계나 머신러닝(이상 탐지, 소요 시간 예측, 용량 예측)도 결국 이렇게 엮인 원자료가 있어야 시작할 수 있습니다.

문제 3. 쓰는 기능은 일부인데 비용은 세 갈래로 나갑니다

스케줄러에서 매일 쓰는 기능은 정해진 시각 실행, 선후행, 재시도, 시간 초과, 원격 실행, 실행 로그 정도입니다. 그런데 비용은 제품 단위로 나갑니다. 제품 밖의 기능이 필요하면 벤더 개발이나 별도 연동을 기다려야 합니다.

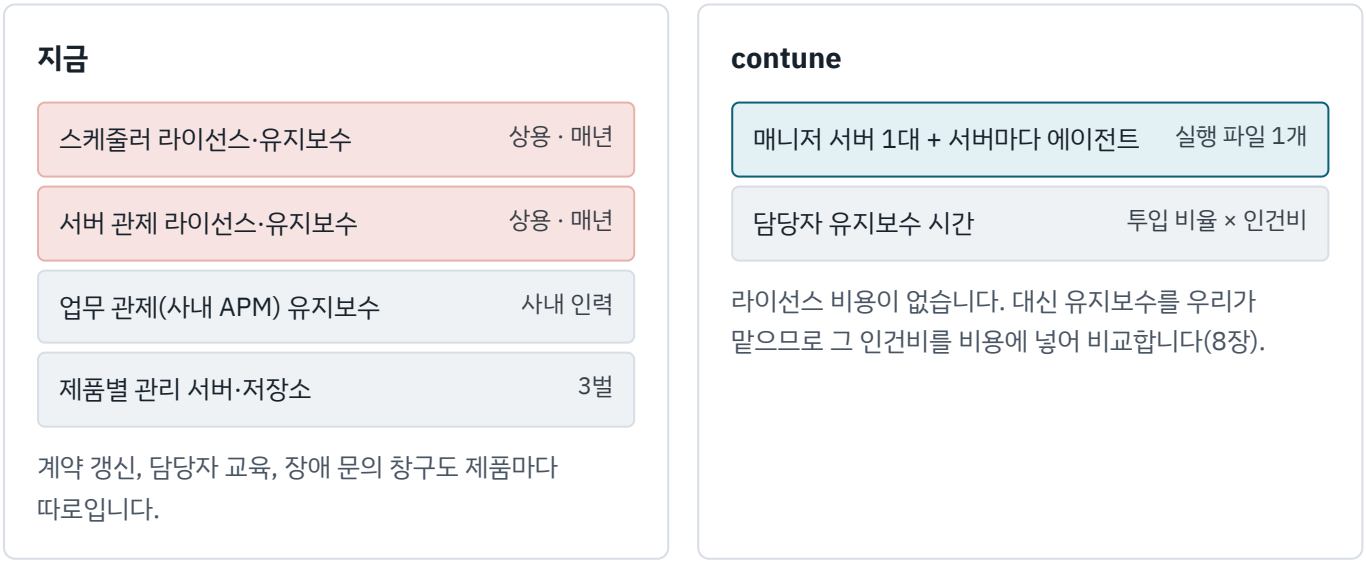


그림 5 비용 구조 비교

03 왜 세 도구를 하나로 합쳐야 하나

합치는 목적은 화면을 모으는 데 있지 않습니다. 모든 기록을 **번호 하나**로 잇는 데 있습니다. 그리고 그 번호는 **프로세스를 띄우는 순간에만** 붙일 수 있습니다.

택배 송장번호와 같습니다. 물건을 보내는 순간 붙여야 이후의 이동 기록이 모두 그 번호로 조회되고, 배송이 끝난 뒤에는 붙일 수 없습니다. 스케줄러가 배치를 띄울 때 번호를 넘겨야 그 배치가 띄운 서비스의 자원·업무 지표·함수 기록이 같은 번호를 달고 쌓입니다. 그래서 **띄우는 쪽(스케줄러)과 재는 쪽(관제)이 한 제품이어야 합니다.**



→ 모두 **같은 번호, 같은 시간축, 같은 DB**. 화면에서는 클릭 한 번, SQL에서는 조인 한 줄로 이어집니다.

그림 6 실행 번호가 기록을 잇는 과정

도구별로 아는 것과 모르는 것

	무엇을 실행했나	어느 프로세스가 자원을 썼나	처리가 밀렸나	어느 함수가 CPU를 썼나	서로 이어지나
배치 스케줄러	○	×	×	×	× 시각으로 추정
서버 관제	×	○	×	×	
업무 관제	×	×	○	×	
contune	○	○	○	○	○ 실행 번호

"장애는 온라인에서 나는데 왜 스케줄러까지 합치나요?"

장중 체결 서비스를 띄우는 것이 새벽 개장 준비 플로우이고, 그 실행이 번호의 출발점입니다. 반대 방향으로도 씁니다. 업무 지표가 같은 DB에 있으므로 "체결 대기가 0건이 된 뒤에 정산 시작" 같은 조건을 스케줄에 바로 겁니다(당일 여유가 있으면 추가하는 확장 기능).

04 왜 자체 구현인가

같은 목표를 이루는 길을 네 가지로 놓고 비교했습니다. 판단 기준은 3장의 **실행 번호로 잇기**가 되는지, 그리고 데이터를 우리가 쥐는지입니다.

선택지	실행 번호로 잇기	함수 단위 상시 기록	데이터를 SQL로 직접	비용	폐쇄망·기능 추가
A. 지금 도구 유지 + 연동 개발	△ 제품끼리 번호를 주고받지 못해 시각·서버 이름으로 맞춤	×	△ 제품별 형식, 꺼내는 범위 제한	높음 기존 비용 + 연동비	△ 벤더 일정에 묶임
B. 오픈소스 조합 스케줄러 + 관제 + 프로파일러	△ 셋을 잇는 코드를 결국 직접 짜야 함	△ 별도 도구, 따로 저장	○ 단, 저장소 3개	낮음	△ 제품 3개를 익히고 운영
C. 대형 통합 관측 제품 상용 APM·관측 플랫폼	△ 서비스·서버는 잇지만 배치 스케줄러가 없음	△ 제품에 따라 가능	△ 벤더 저장소, 외부 반출 우려	매우 높음	△ 폐쇄망 도입 제약
D. 자체 구현 (contune)	○ 띄우는 쪽이 번호를 심음	○ 같은 번호, 같은 DB	○ 공개 스키마 DB 1개	낮음 서버 + 담당자 시간	○ 코드가 사내에 있음

B를 택해도 세 제품을 잇는 데이터 모델과 연동 코드를 직접 만들어야 합니다. 이 과제의 핵심이 바로 그 부분이므로, 처음부터 하나로 만드는 편이 단순합니다.

자체 구현을 감당할 수 있는 이유

어려운 부분은 검증된 부품에 맡깁니다

함수 추적은 리눅스 커널에 들어 있는 `perf`, 저장은 SQLite, 실행 시각 해석은 검증된 cron 라이브러리를 씁니다. 우리가 짜는 코드는 이것들을 잇는 부분입니다.

구조가 작습니다

Go로 만든 실행 파일 1개, 외부 라이브러리 4개. 연결은 에이전트에서 매니저 쪽 한 방향뿐이라 방화벽은 매니저 포트 1개만 엽니다.

바꿀 자리를 미리 나눠 둡니다

수집, 프로세스 추적, 함수 추적, 지표 저장, 상태 저장, 알림을 인터페이스 6개로 나눴습니다. 저장소를 대용량 DB로 바꾸거나 자바 추적을 붙일 때 그 부분만 바꿉니다.

자체 구현은 유지보수를 우리가 맡는다는 뜻입니다. 그래서 비용 비교에 담당자 인건비를 넣었고, 하루 동안의 코드 규모와 개발 기록을 발표 때 공개합니다.

05 contune의 구성

서버마다 에이전트 1개를 두고, 매니저 1대가 실행을 지시하고 기록을 모읍니다. 둘 다 같은 실행 파일입니다.

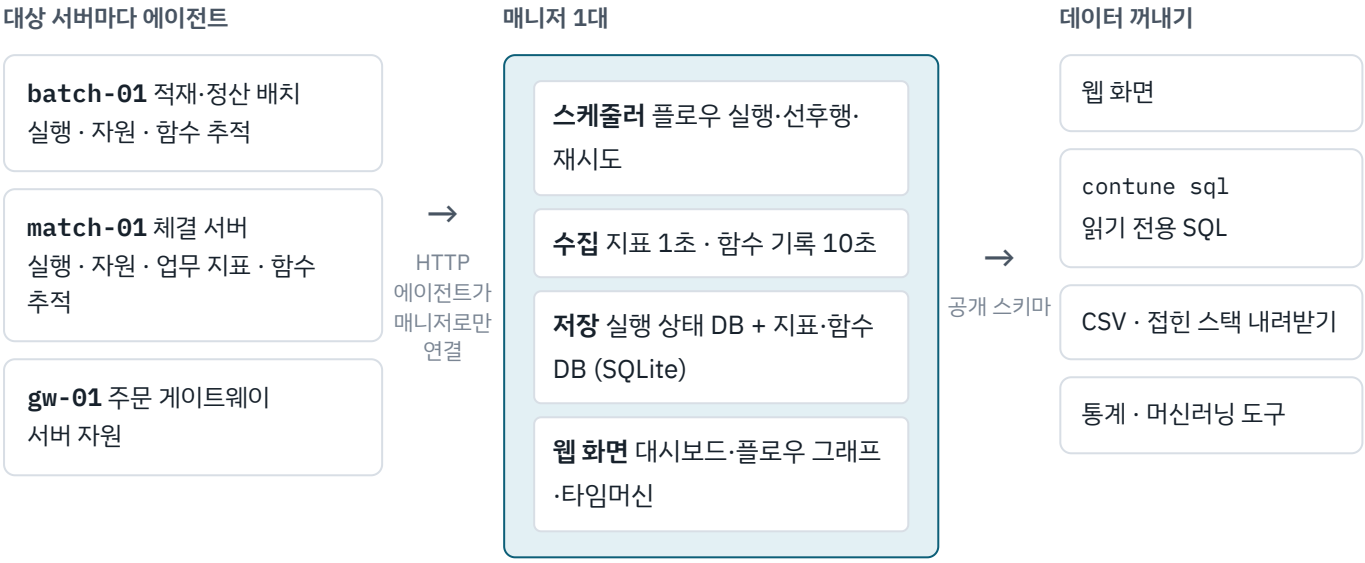


그림 7 구성도. 실행 명령도 에이전트가 매니저에게 가져가므로 대상 서버에 들어오는 포트를 열 필요가 없습니다.

영역	하는 일	지금 맡는 도구
배치 스케줄·실행	정해진 시각에 플로우 실행, 선후행(병렬·합류·서버 넘기), 재시도, 시간 초과. 스크립트가 띄운 서비스 프로세스까지 실행 번호로 추적	배치 스케줄러
서버 관제	서버 CPU·메모리를 1초 단위로 수집	서버 관제
업무 지표	접수·처리 건수, 대기 건수, 응답시간을 서버 지표와 같은 통로로 수집	업무 관제
함수 단위 추적	실행이 띄운 프로세스가 CPU를 쓴 함수를 상시 표본 수집(초당 49회). 끝난 배치와 지나간 장애 구간도 나중에 고름	없음
열린 데이터	공개 스키마, 분석용 뷰, 읽기 전용 SQL, CSV 내보내기	없음

> 함수 추적은 배치를 감싸지 않고 옆에서 붙습니다. 추적이 실패해도 배치와 서비스는 영향을 받지 않고, 추적 대상은 contune이 띄운 프로세스뿐입니다. 기록에는 함수 이름과 표본 수만 남고 메모리 내용이나 주문 데이터는 남지 않습니다.

06 예상 결과물 화면

설계 문서의 화면 계약과 시연 시나리오대로 그린 시안입니다. 시연에서는 체결 서비스(시연용 C 프로그램)에 주문을 1,500건/초에서 6,000건/초까지 몰아 넣어 실제로 체결을 밀리게 만들고, 그 장면을 아래 화면으로 따라갑니다. 화면의 수치는 시연용 합성 부하 기준 예시입니다.

① 대시보드: 서버와 업무를 한 화면에

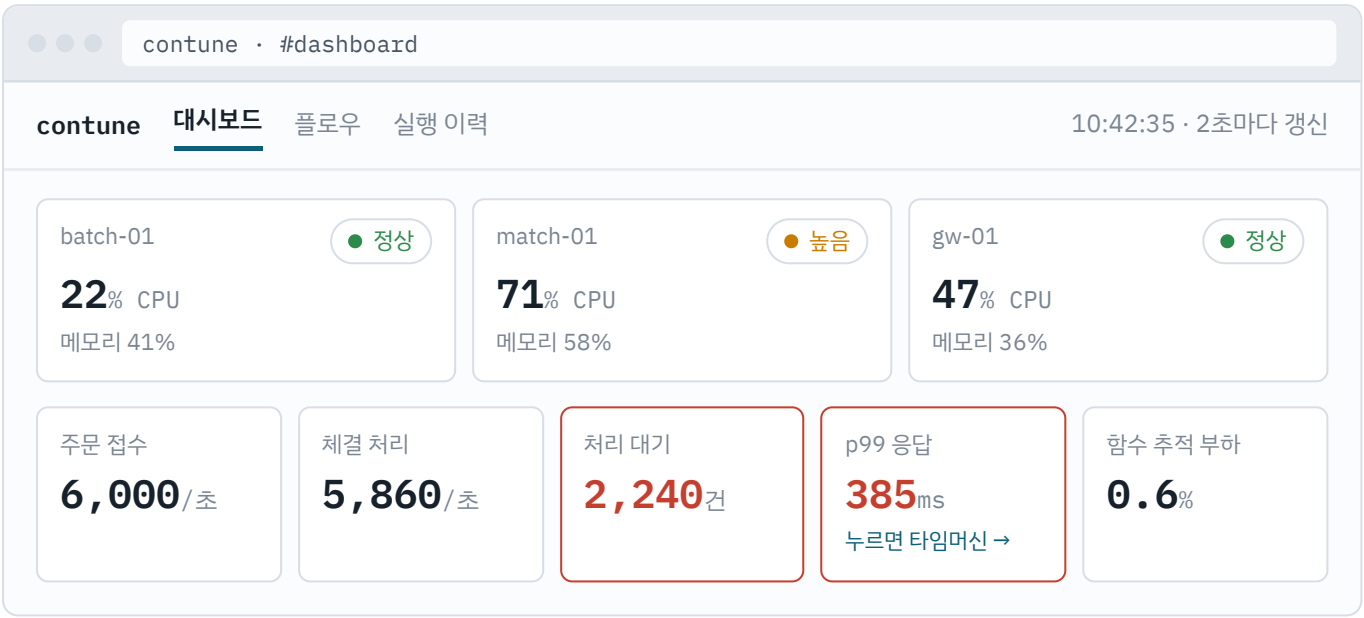
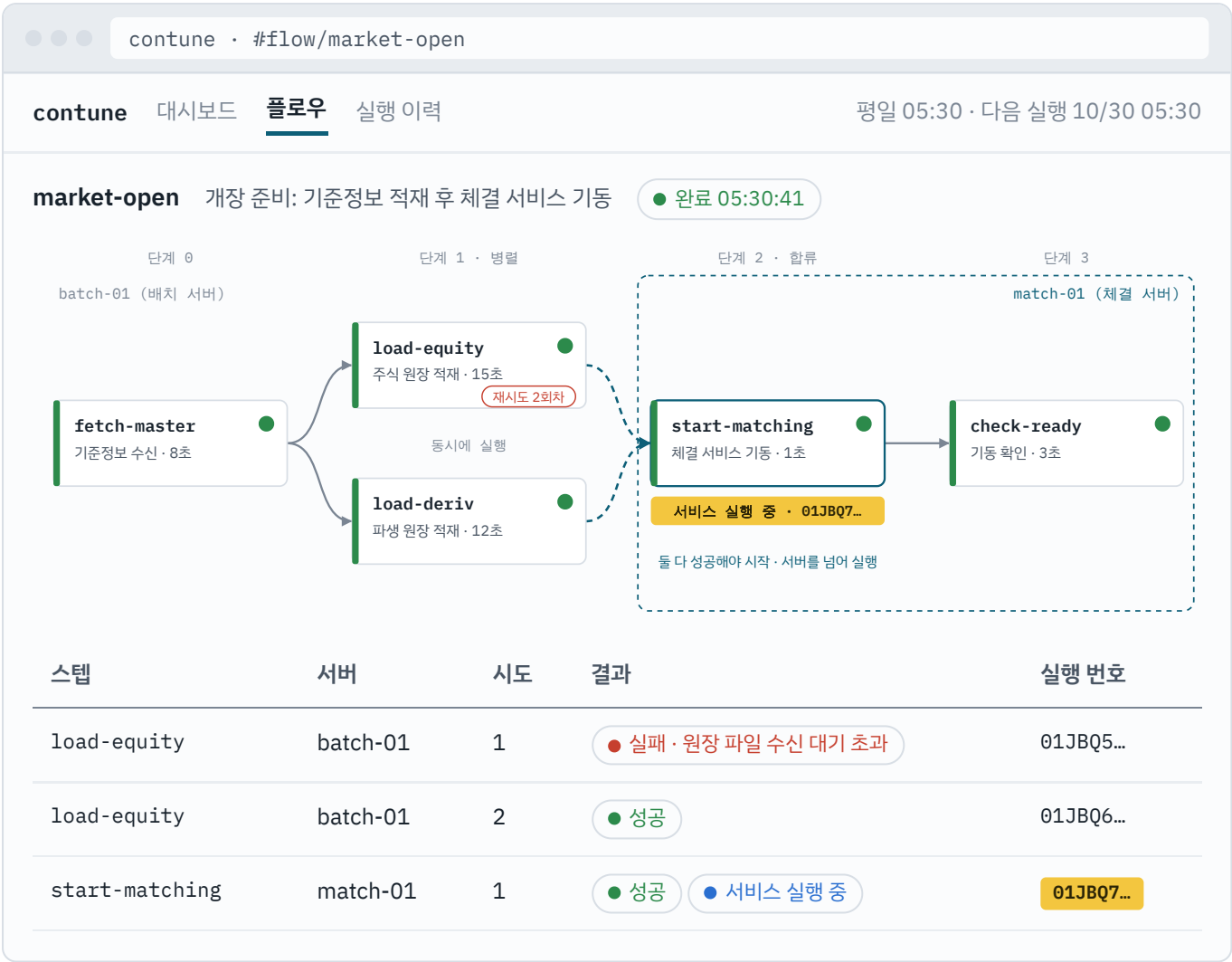


그림 8 업무 지표 타일에는 그 지표를 보낸 서비스의 실행 번호가 붙어 있어, 누르면 바로 그 실행의 타임머신으로 갑니다. 함수 추적이 쓰는 CPU도 숨기지 않고 띄웁니다.

② 플로우 그래프(DAG): 새벽 개장 준비가 어떻게 돌았나



스텝	서버	시도	결과	실행 번호
load-equity	batch-01	1	● 실패 · 원장 파일 수신 대기 초과	01JBQ5…
load-equity	batch-01	2	● 성공	01JBQ6…
start-matching	match-01	1	● 성공 ● 서비스 실행 중	01JBQ7…

그림 9 작업이 갈라지고(병렬) 합쳐지고(합류) 서버를 넘는 흐름을 한눈에 봅니다. 첫 시도가 실패한 스텝은 재시도 배지로 표시하고, 노드를 누르면 그 실행의 타임머신이 열립니다. 체결 서비스를 띄운 스텝은 끝났어도 서비스가 살아 있는 동안 같은 번호로 계속 기록합니다.

③ 타임머신: 그 실행의 기록을 한 시간축에

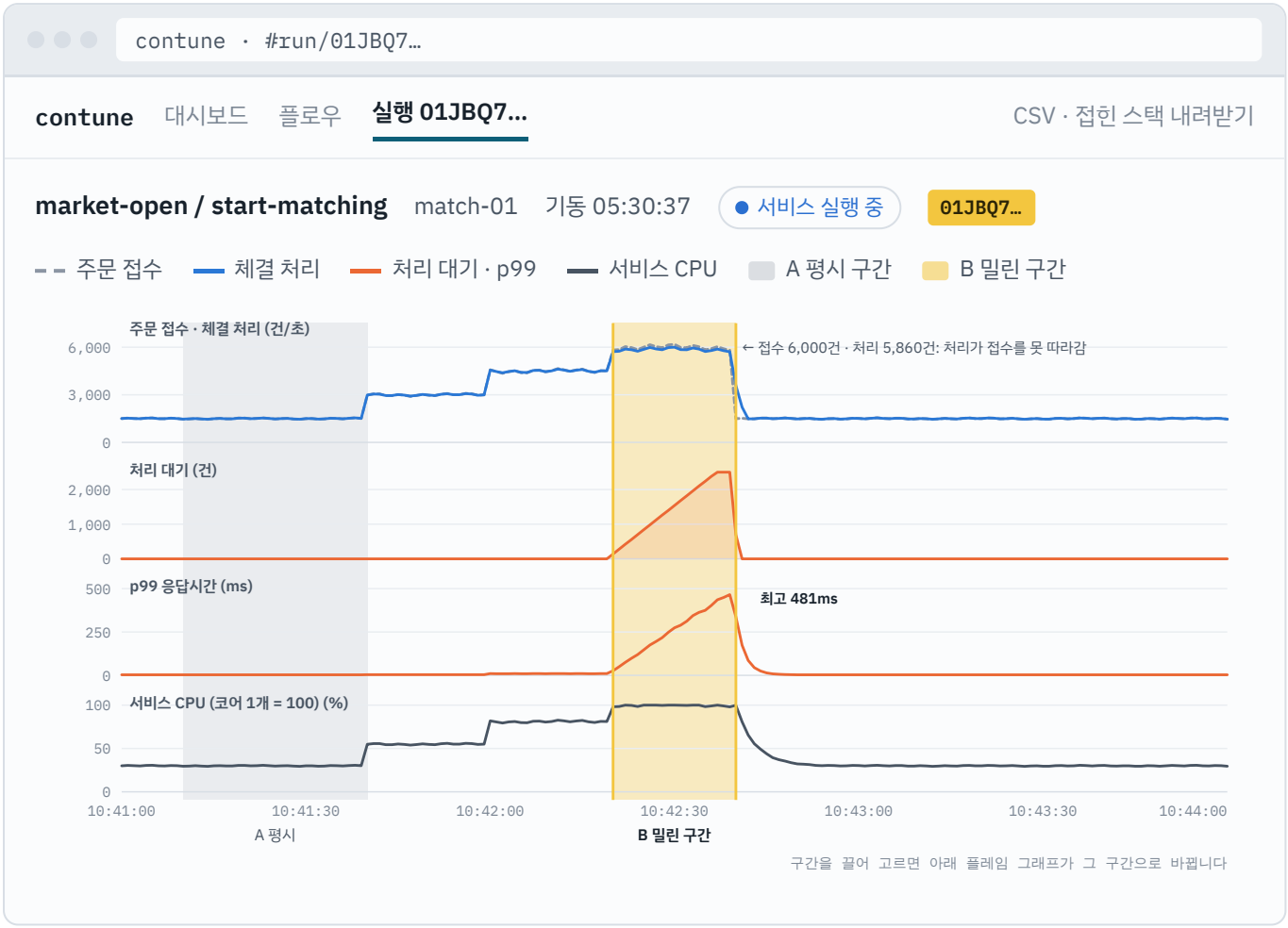


그림 10 네 지표가 같은 실행 번호로 같은 시간축에 쌓입니다. 접수가 처리를 앞지르는 순간 대기와 p99가 오르고 서비스 CPU가 100%에 붙습니다. 차트에서 구간을 끌어 고르면 아래 플레임 그래프가 그 구간으로 다시 그려집니다.

④ 플레임 그래프: 그 구간에 CPU를 쓴 함수

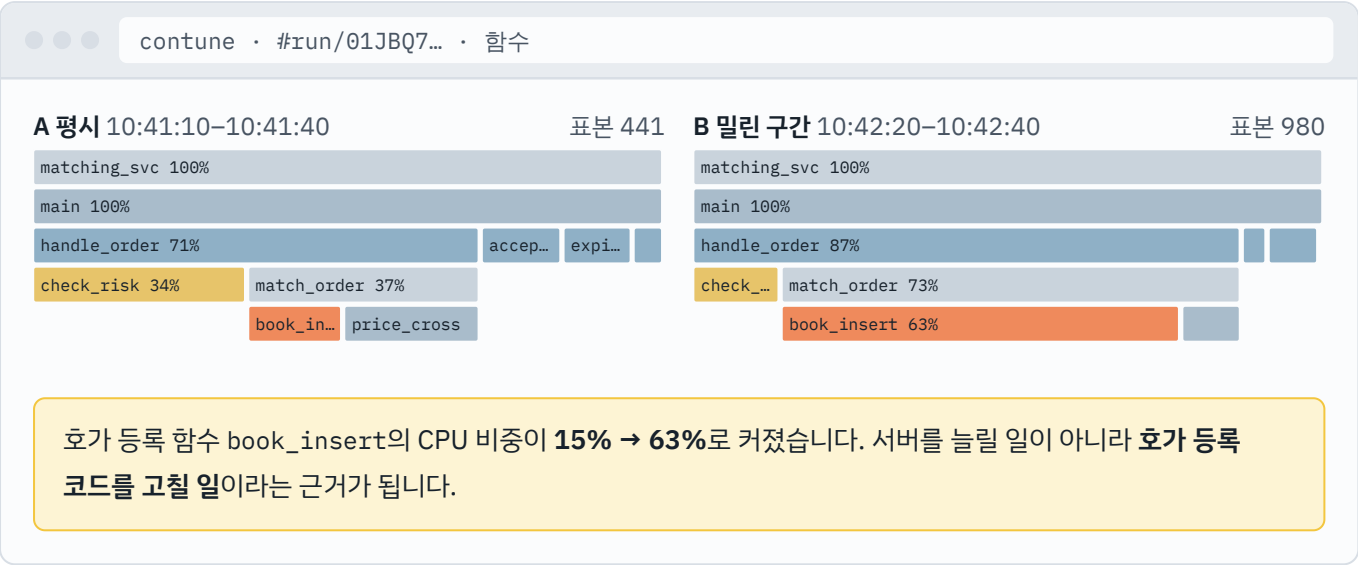


그림 11 위가 바깥 함수, 아래로 갈수록 안쪽 함수입니다. 막대 폭이 CPU를 쓴 비율입니다. 폭주가 끝난 뒤에도 같은 구간을 다시 볼 수 있으므로 재현이 필요 없습니다.

⑤ 데이터 꺼내기: 화면 밖에서 SQL로 바로

```
-- 최근 5영업일, 접수량 구간별 체결 p99: 초당 몇 건부터 밀리기 시작하나
$ contune sql "SELECT in_tps, seconds, avg_p99_ms FROM ..."
 in_tps | seconds | avg_p99_ms
--------+---------+-----------
   1500 | 109,560 |        3.4
   3000 |   6,420 |        4.1
   4500 |   1,140 |        9.8
   6000 |     180 |      254.1

-- 밀린 구간과 평시에 CPU를 쓴 함수 비중(%): 업무 지표와 함수 기록을 실행 번호로 조인
$ contune sql --csv "WITH w AS (...) SELECT leaf, normal_pct, slow_pct ..." > slow.csv
 leaf           | normal_pct | slow_pct
----------------+------------+---------
 book_insert    |       15.2 |     63.4
 check_risk     |       34.0 |     14.0
 price_cross    |       21.8 |      9.6
 expire_orders  |       11.1 |      8.2
```

그림 12 같은 데이터를 읽기 전용 SQL과 CSV로 꺼냅니다. 첫 질의는 증설 판단의 근거, 두 번째 질의는 코드 수정의 근거입니다. 두 질의는 설계 단계에서 같은 스키마의 합성 데이터로 실행해 확인했습니다(수치는 예시).

07 하루, 2명이 만들 수 있는가

대회 규칙상 코드는 당일 처음부터 씁니다. 그래서 사전에는 **설계 문서, 작업카드, AI 지시문**만 준비했고, 당일에는 2명이 각자 AI 코딩 세션 2개씩, 모두 4개를 병렬로 돌립니다. 사람은 계약(데이터 형식과 API)과 검수를 맡고 AI 세션은 구현을 맡습니다.



그림 13 당일 일정. 15:00에 개념 7개가 최소 형태로 한 바퀴 도는지 점검하고, 이후에는 범위만 넓힙니다.

15:00까지 반드시 도는 개념 7개

넓이(화면 수, 기능 수)는 줄여도 개념의 고리는 줄이지 않습니다. 하나라도 빠지면 검증판이 아닙니다.

개념	최소 형태	점검 방법
스케줄	플로우 2개, 선후행(병렬·합류·서버 넘기), 재시도, 시간 초과, 플로우 그래프	수동 실행 → 스텝 5개가 순서대로 성공, 적재 스텝은 2회차에 성공
원격 실행	명령 큐, 프로세스 그룹 실행, 이벤트 재전송, 서비스 프로세스 추적	실행 중 매니저를 재시작해도 결과가 반영됨
한 통로 수집	서버·업무 지표 1초, 함수 기록 10초, SQLite 저장, 조회 API	대시보드에 서버 3대와 업무 지표
실행 번호 연결	타임머신, 업무 지표 타일에서 실행으로 이동	p99 타일 클릭 → 자원과 업무 지표가 한 차트에
함수 단위 추적	perf 표본 수집, 플레임 그래프, 구간 고르기, 부하 지표	밀린 구간의 `book_insert` 비중이 평시보다 큼, 부하 1% 미만
데이터 주권	분석용 뷰, 읽기 전용 SQL, CSV 내보내기	그림 12의 질의 2개가 결과를 냄
보안 최소선	에이전트 토큰, 검토된 YAML의 명령만 실행, 일반 사용자 권한	토큰 없는 요청은 거부(401)

이미 확인한 것

항목	결과
운영 서버 perf 사용	사용 가능 확인
운영과 같은 조건의 C 빌드	최적화 빌드(`-O2`, 프레임 포인터 없음)에서도 dwarf 방식으로 호출 경로 `main → handle_order → match_order → book_insert`를 정확히 복원
추적 부하	10초 구간당 약 0.06초 → 코어 1개의 약 **0.6%**. 대상 프로그램 소요 시간은 측정 오차 안(13.3~13.4초 → 13.4초)
스크립트를 벗어난 서비스 추적	`setsid`로 띄워 스크립트 밖으로 나간 프로세스를 환경변수 표식으로 찾음(일반 사용자 권한)
분석 질의	그림 12의 SQL 2개를 같은 스키마의 합성 데이터로 실행해 확인
시연 컨테이너	Docker 기본 설정은 perf를 막지만 `PERFMON` 권한 하나로 일반 사용자 실행 확인

시연에서 실제로 동작하는 것과 시연용인 것

실제로 동작

스케줄 실행, 원격 실행, 서비스 프로세스 추적, perf 함수 기록, 수집·저장·조회, 화면, SQL·CSV 꺼내기. 매니저 1대와 에이전트 3개를 컨테이너로 띄웁니다.

시연용으로 만든 것

체결 서비스는 합성 주문으로 부하를 만드는 C 프로그램(약 200줄)이고, 함수 이름과 수치도 시연용입니다. 컨테이너는 서버별 자원 값이 다르게 보이지 않아 서버 자원만 시뮬레이터로 만듭니다. 실서버에서는 수집기 설정 한 줄만 바꿉니다.

범위 밖으로 둔 것과 늦어질 때의 대응

검증판에서 하지 않음	웹 로그인·권한, 암호화 통신(TLS), 이중화, 휴일 캘린더, 운영자 강제 성공 처리, 자바 함수 추적, 대기 시간 분석, 제품 안의 머신러닝. 모두 인터페이스 자리만 두고, 운영 투입 전에 갖춰야 할 항목으로 정리했습니다.
15:00에 늦으면	① 대시보드를 서버 카드와 업무 타일만으로 ② CSV 내보내기를 뒤로 ③ 플레임 그래프의 구간 고르기를 빼고 화면 전체 범위로 ④ 플레임 그래프 대신 상위 함수 표. **플로우 그래프와 함수 기록 수집은 줄이지 않습니다.**
시연 장비 문제	perf 권한은 사전에 한 줄 명령으로 점검합니다. 15:00 점검 직후와 코드 동결 뒤 두 번 녹화본을 만듭니다. Docker가 막히면 단일 프로세스 모드(`contune dev`)로 같은 시연을 합니다.

08 기대 효과

시간

지금 도구 3개, 5단계. 원인 코드는 재현 뒤에야

이후 **화면 1개, 2단계. 지나간 장애도 함수까지**

판단도 빨라집니다. "서버를 늘릴지, 코드를 고칠지"를 같은 화면에서 근거로 가릅니다.

데이터

지금 제품 화면과 보고서 범위 안에서만

이후 **운영 데이터 5종을 SQL 한 번으로 조인**

실행 이력, 서버 자원, 배치·서비스 자원, 업무 지표, 함수 기록. 소요 시간 이상 탐지, 배치 종료 시각 예측, 서버 용량 예측, 배포 전후 성능 회귀 탐지에 바로 씁니다.

비용

지금 상용 계약 2건 + 사내 APM 유지보수

이후 **매니저 서버 1대 + 담당자 시간**

필요한 기능은 벤더 일정과 상관없이 직접 붙입니다. 코드가 사내에 있습니다.

비용 비교 계산식

> 연간 절감액 = (스케줄러 라이선스·유지보수 + 서버 관제 라이선스·유지보수 + 사내 APM 유지보수 인건비) − (매니저 서버 비용 + contune 담당자 투입 비율 × 인건비)

항목	지금 (연간)	contune (연간)
스케줄러 라이선스·유지보수	금액 또는 비율 기입	0
서버 관제 라이선스·유지보수	금액 또는 비율 기입	0
업무 관제(사내 APM) 유지보수	투입 인력 기입	contune 유지보수에 포함
관리 서버	제품별 3벌	매니저 1대
자체 유지보수 인력	-	투입 비율 기입

검증판 이후

해커톤 결과물은 개념 검증판입니다. 운영에 쓰려면 7장에서 범위 밖으로 둔 로그인·권한, 암호화 통신, 이중화, 휴일 캘린더를 갖춰야 합니다. 이 항목들은 모두 설계에서 자리를 정해 두었으므로, 이후 작업은 정해진 자리에 구현을 채우는 일입니다. 대상 서버가 수십 대를 넘으면 지표 저장소만 시계열 전용 DB로 바꿉니다.

이 문서의 화면은 설계 시안이며, 수치는 시연용 합성 부하 기준 예시입니다. 사전 실측 값은 7장에 출처와 조건을 함께 적었습니다.